Graph Preconditioning for High-Dimensional Fixed Effects Regression

Alexander Fischer[1] and Kristof Schröder[2]

Draft: August 8, 2026

Abstract. The Method of Alternating Projections (MAP) is the canonical algorithm for estimating high-dimensional fixed-effect regressions. It is fast when absorbed factors are well connected, but converges slowly on sparse or nearly nested fixed-effect graphs, such as matched employer-employee panels where worker-firm mobility links separate worker and firm effects. The convergence behavior of MAP depends on the mobility pattern linking workers to firms, yet MAP only indirectly makes use of this information by iterating over one fixed effect at a time. That mobility pattern is, however, directly encoded in the matrix of worker-firm match counts, which together with the worker and firm count diagonals forms a graph Laplacian after a sign flip and admits sparse approximate Cholesky factorization. We propose a graph-preconditioned Krylov solver whose reusable preconditioner is built from small, local factor-pair subproblems - worker-firm, worker-year, and so on - that use the graph directly. Benchmarks show that all methods are fast on well-connected designs. As connectivity weakens, MAP and diagonally preconditioned LSMR slow down, while factor-pair preconditioning remains fast and its runtime falls because the graph preconditioner is cheaper to construct on sparser graphs.

Keywords: high-dimensional fixed effects; alternating projections; preconditioning; matched employer-employee data; computational econometrics

JEL codes: C55; C63; C81; C87; J31

1. Introduction

Fixed-effect regressions are ubiquitous in applied econometrics: according to Goldsmith-Pinkham (2026), roughly half of published research in top economics and finance journals mentions “fixed effects”. They appear throughout all fields of applied economics: labor economists use worker and firm fixed effects to separate worker heterogeneity from firm wage premia; health economists study physician practice styles with individual-physician and region fixed effects in mover designs; and education researchers study models with school, student, teacher, or student-teacher fixed effects.

The standard computational starting point for estimating these regressions efficiently is the Frisch-Waugh-Lovell (FWL) theorem (Frisch and Waugh 1933; Lovell 1963). FWL reduces the fixed-effect estimation problem to “residualizing” the outcome and every regressor of interest against the fixed effects, and then to running a low-dimensional regression on the residualized variables.

The workhorse method for these fixed-effect residualizations is the Method of Alternating Projections (MAP), also known as iterative demeaning or the “Zig-Zag” algorithm (Guimaraes and Portugal 2010;

[1]trivago
[2]appliedAI Institute for Europe gGmbH

Gaure 2013). Most leading software implementations of fixed-effect regression, such as `reghdfe` in Stata (Correia 2014; 2017), `fixest` (Bergé et al. 2026) in R, or PyFixest in Python (PyFixest Developers 2026), use MAP or MAP-based variants with acceleration.

Because the AKM worker-firm model is among the most prominent applications of high-dimensional fixed effects, we use a worker-firm-year panel based on Abowd et al. (1999) as the running example in this paper. The method of alternating projections cycles through the fixed-effect dimensions one at a time: it first subtracts worker means, then firm means, then year means, and repeats until convergence. The coupling between fixed effects is never used directly; the cross-fixed effects information is transmitted only indirectly through the residual that each step hands to the next. How fast MAP converges therefore depends on how quickly information about this coupling can propagate from one update to the next.

Fixed effects and their coupling form a graph structure. In a worker-firm panel, movers create paths between firms, while stayers add observations without connecting firms to one another. When this graph is sparse or poorly connected, some fixed-effect directions are nearly collinear, and MAP needs many iterations to propagate information across it before converging. The same graph features that determine whether worker and firm effects are identified also govern MAP convergence. These features include worker mobility, sorting, and segmentation into disconnected labor markets with thin bridges, such as public- and private-sector employment when workers only rarely move between the two sectors.

The graph structure of the fixed effects can be algebraically encoded in the off-diagonal blocks of the fixed-effect Gramian (Correia 2017). These off-diagonal blocks record co-occurrences among the fixed effects: which workers work at which firms, which physicians practice in which regions, or which families move across counties.

In this paper, we propose a new preconditioner that directly encodes the co-occurrence graph. A preconditioner is a cheap approximation to the system being solved; supplied to an iterative solver, it reduces the number of iterations without changing the solution. We build ours from local factor-pair subproblems that use the graph structure of the Gramian: for example, a worker-firm subproblem incorporates the observed links between workers and firms. A preconditioner is only useful if it approximates the inverse of the co-occurrence Gramian at low cost. We obtain such an approximation by exploiting the fact that, after a sign change, each factor-pair block is a graph Laplacian, which admits sparse approximate inverses following ideas from the Laplacian-solver literature (Spielman and Teng 2014; Gao et al. 2025). The preconditioned system is then solved with a Krylov solver, an iterative method for large linear systems.[3]

Figure 1 plots full-regression runtime as worker-firm connectivity varies, with well-connected graphs on the left and weakly connected graphs on the right. The left panel compares package defaults; the right panel holds the PyFixest code path and accuracy target fixed while varying the preconditioner.

[3] The Julia implementation of fixed-effect regression, `FixedEffectModels.jl` (FixedEffectModels.jl Developers 2026), uses the same Krylov solver as we do - LSMR (Fong and Saunders 2011) - but only with diagonal preconditioning, which ignores the off-diagonal co-occurrence structure entirely; our contribution is the preconditioner, not the use of LSMR for the outer iteration.

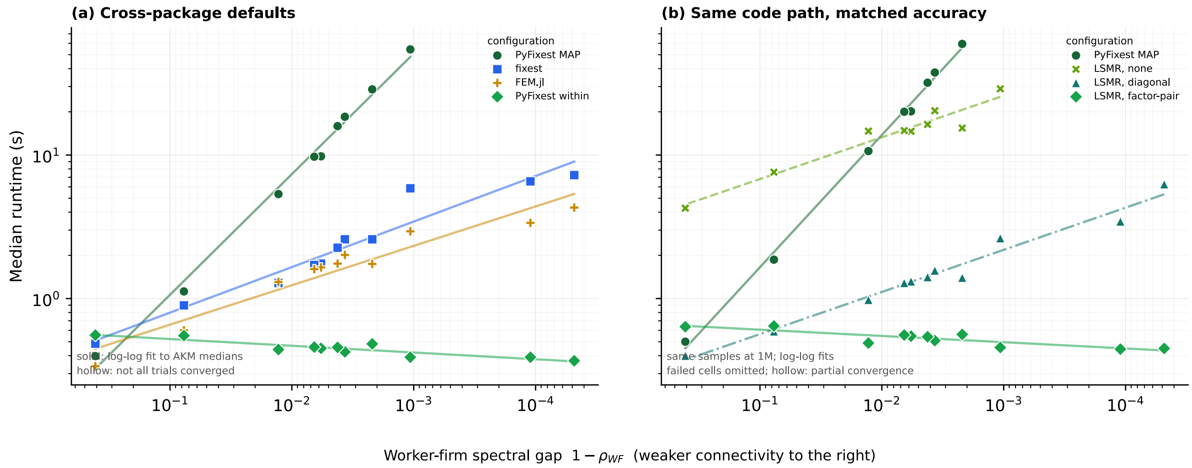

Figure 1: Median runtime against the worker-firm spectral gap, on log-log axes. Smaller gaps mean weaker connectivity, so the horizontal scale decreases from left to right. Both panels report simulated worker-firm-year panels with 1 million observations, in which worker mobility and sorting vary. Each point is the median of three fits on one stored sample; packages apply their own default singleton treatment. Panel (a) compares package defaults. Panel (b) fixes the PyFixest code path and achieved accuracy, separating preconditioner effects from language and package differences. Solid lines are log-log fits to the medians. Failed cells are omitted; hollow markers indicate partial convergence.

At high connectivity, all implementations finish quickly. As the spectral gap narrows, MAP and LSMR without factor-pair preconditioning slow down. Factor-pair LSMR remains fast and becomes faster in the least connected mobility designs because the worker-firm subproblems are cheaper to construct.

The rest of the paper is organized as follows. Section 2 sets up the fixed-effect absorption problem, and Section 3 introduces the AKM model as our running example. Section 4 develops the graph structure of the fixed-effect Gramian, and Section 5 connects this structure to the convergence behavior of MAP. Section 6 introduces preconditioning and then constructs the factor-pair Schwarz preconditioner. Section 7 reports the runtime benchmarks; Section 8 describes the software through which the new algorithm is available; and Section 9 concludes. Appendix B reports numerical equivalence, memory use, and additional benchmark diagnostics.

2. Absorbing Fixed Effects[4]

We focus on the linear model

$$y = X\beta + D\alpha + \varepsilon, \tag{1}$$

where X contains the regressors of interest, D is the fixed-effect design matrix, and α collects the fixed-effect coefficients. In high-dimensional applications D may have hundreds of thousands or millions of columns, and forming or inverting the full system in $[X \quad D]$ might prove computationally infeasible. Fortunately, via the Frisch-Waugh-Lovell (FWL) theorem, we can compute $\hat{\beta}$ without ever forming $[X \quad D]$ or inverting its cross product.

FWL reduces the computation of $\hat{\beta}$ to two steps. In step one, we residualize the outcome and each covariate against the fixed effects: we regress y on D and keep the residual $\tilde{y}$, and we do the same for every column

[4]Researchers employ several names for this operation: "absorbing fixed effects", "demeaning", "residualizing", or applying the "within transformation". We use these terms interchangeably throughout.

of X. Denoting M_D as the linear operator that sends a variable to this residual, so that $\tilde{y} = M_D y$ and $\tilde{X} = M_D X$, the coefficient of interest is recovered by regressing the residualized outcome on the residualized covariates,

$$\tilde{y} = M_D y, \quad \tilde{X} = M_D X, \quad \hat{\beta} = \left(\tilde{X}' W \tilde{X}\right)^{-1} \tilde{X}' W \tilde{y}, \tag{2}$$

where W is a diagonal matrix of weights. With one covariate, the procedure consists of three regressions: y on D, x on D, and $\tilde{y}$ on $\tilde{x}$. FWL implies that the slope in the last regression equals the coefficient on x in the full regression of y on x and D.

The residualization step is the weighted least-squares projection of the outcome and each covariate in X onto the column space of the fixed-effect dummy matrix D. For a right-hand side μ, either y or one column of X, we solve

$$\hat{\alpha}_\mu = \arg\min_\alpha \| D\alpha - \mu \|_W^2, \tag{3}$$

and the residual is $\tilde{\mu} = \mu - D\hat{\alpha}_\mu$. The first-order condition for Equation 3,

$$D' W \left(D\hat{\alpha}_\mu - \mu\right) = 0, \quad \text{equivalently} \quad G\hat{\alpha}_\mu = D' W \mu, \quad G = D' W D, \tag{4}$$

determines $\hat{\alpha}_\mu$. Each FWL residualization uses the same coefficient matrix G; only the right-hand side changes as we move from the outcome to the covariates. The cost of residualization therefore depends on the structure of the Gramian G. In the next section, we illustrate this structure with the AKM worker-firm model and explain why fixed-effect designs have a direct graph interpretation. Sections 5–6 then return to the algorithms and show how the structure of the Gramian and its associated graph governs MAP convergence, and explain how we use it to construct the factor-pair Schwarz preconditioner.

3. A Running Example: The AKM Model

The AKM model of Abowd et al. (1999) introduces the worker-firm setting used throughout the paper. It separates persistent worker heterogeneity from firm wage premia using workers who move across firms. We write the AKM regression equation as

$$y_{it} = \alpha_i + \psi_{J(i,t)} + \varphi_t + x_{it}' \beta + \varepsilon_{it}, \tag{5}$$

where α_i is a worker fixed effect, $\psi_{J(i,t)}$ is the fixed effect for the firm employing worker i at time t, and φ_t is a time fixed effect.

The AKM specification has a natural graph representation. Workers and firms are nodes in a bipartite graph, and each employment spell contributes an edge. Worker moves induce a firm-to-firm graph, in which two firms are connected when at least one worker is observed at both firms. Although stayers - workers who never change their employer - add observations to existing worker-firm links, they do not create bridges between firms. Year effects enter as a third, low-dimensional factor observed on the same worker-firm records. Figure 2 contrasts a well-connected mobility graph with one that fragments under strong sorting.

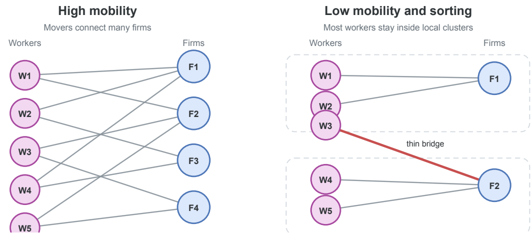

Figure 2: Worker-firm graph connectivity. When mobility is high, many paths connect firms. With low mobility and strong sorting, the graph breaks into nearly separate clusters joined only by narrow bridges.

These mobility links matter both for identification and, as we will argue later, for computation. In AKM, worker and firm fixed effects are separately identified through movers, who provide the comparisons that distinguish worker heterogeneity from firm wage premia. A worker observed at only one firm provides no such comparison: a high wage could reflect an unusually productive worker, a high-wage firm, or both. Without worker moves, these components are not separately identified. Movers observed across firms with different wage profiles help attribute wage variation to worker effects or firm premia. If a worker earns high wages across several firms, the comparison points toward a worker effect; if many different workers earn higher wages at the same firm, it points toward a firm premium. The more such cross-firm comparisons the data contain, especially across otherwise different firms, the easier it is to identify worker and firm premia. In the graph, these moves are exactly the edges that connect firms; additional moves of workers across firms add worker-firm links to the graph.

4. The Graph Structure of the Gramian

The bipartite graph of worker and firm connections introduced in Section 3 has an algebraic representation in the block structure of the Gramian $G = D'WD$ (Correia 2017). Suppose that the columns of D are ordered as worker levels, firm levels, and year levels. Then

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}. \tag{6}$$

The diagonal blocks G_{WW}, G_{FF}, and G_{YY} contain weighted counts for workers, firms, and years. An observation belongs to one level of each factor, so these blocks are diagonal; solving them requires only division by group counts.

The off-diagonal blocks are cross-tabulations: the worker-firm block C_{WF} records how often worker i is observed at firm j, and the worker-year and firm-year blocks have analogous interpretations.

As a small example, we construct a worker-firm panel and populate its Gramian. For simplicity, we ignore any regression weights and set $W = I$.

Obs.	Worker	Firm	Year	y
1	W_1	F_1	Y_1	3.2
2	W_1	F_2	Y_2	4.1
3	W_2	F_1	Y_1	2.8
4	W_2	F_1	Y_2	3.9
5	W_3	F_2	Y_1	5.0

Obs.	Worker	Firm	Year	y
6	W_3	F_2	Y_2	4.5

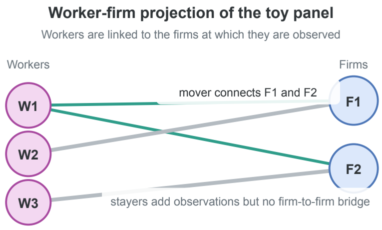

Figure 3: Worker-firm projection of the example panel. Worker W_1 is a mover; workers W_2 and W_3 are stayers.

Figure 3 plots the worker-firm projection of this panel. Worker W_1 has employment spells both in F_1 and F_2 and creates a link between the two firms; in AKM terms, W_1 is a mover. Worker W_2 stays at F_1 for two periods, and W_3 stays at F_2 for two periods. Both are stayers.

The diagonal blocks are count matrices. In this example, each worker is observed twice, each firm three times, and each year three times, so

$$G_{WW} = \begin{pmatrix} 2 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 2 \end{pmatrix}, \quad G_{FF} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}, \quad G_{YY} = \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix}. \tag{7}$$

The off-diagonal blocks are cross-tabulations between factors. The worker-firm block is

$$C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}. \tag{8}$$

The first row indicates that worker W_1 appears once at firm F_1 and once at firm F_2. Workers W_2 and W_3 are stayers, appearing twice at F_1 and F_2, respectively. The worker-year block is

$$C_{WY} = \begin{pmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 1 \end{pmatrix}. \tag{9}$$

Each worker is observed once in each year. The firm-year block is

$$C_{FY} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}. \tag{10}$$

Firm F_1 appears twice in year Y_1 and once in year Y_2; firm F_2 has the opposite pattern. Combining the diagonal count blocks and the off-diagonal cross-tabulation blocks yields the full Gramian.

With column order $(W_1, W_2, W_3, F_1, F_2, Y_1, Y_2)$, the full Gramian is

$$G = \left(\begin{array}{ccc|cc|cc} 2 & 0 & 0 & 1 & 1 & 1 & 1 \\ 0 & 2 & 0 & 2 & 0 & 1 & 1 \\ 0 & 0 & 2 & 0 & 2 & 1 & 1 \\ \hline 1 & 2 & 0 & 3 & 0 & 2 & 1 \\ 1 & 0 & 2 & 0 & 3 & 1 & 2 \\ \hline 1 & 1 & 1 & 2 & 1 & 3 & 0 \\ 1 & 1 & 1 & 1 & 2 & 0 & 3 \end{array}\right). \tag{11}$$

The worker-firm submatrix stores the bipartite graph algebraically. The diagonal entries are worker and firm counts, and the entries of C_{WF} are edge multiplicities between workers and firms. After flipping the sign of C_{WF}, this submatrix is a graph Laplacian: its off-diagonal entries are non-positive, and every row sums to zero because each diagonal count cancels the off-diagonal spell counts in the same row. For a worker, the diagonal entry is the number of that worker's employment spells, while the off-diagonal entries count how those spells are distributed across firms; firm rows have the analogous interpretation with spell counts summed over workers.

$$L_{WF} = \left(\begin{array}{ccc|cc} 2 & 0 & 0 & -1 & -1 \\ 0 & 2 & 0 & -2 & 0 \\ 0 & 0 & 2 & 0 & -2 \\ \hline -1 & -2 & 0 & 3 & 0 \\ -1 & 0 & -2 & 0 & 3 \end{array}\right). \tag{12}$$

The same Laplacian construction applies to any pair of fixed effects, and the preconditioner of Section 6 builds on these pairwise Laplacians. Before turning to it, however, we introduce the method of alternating projections, which avoids forming the full Gramian G by working only on the diagonal worker, firm, and year blocks.

5. Alternating Projections and Graph Connectivity

The workhorse algorithm for multi-way fixed effects is the Method of Alternating Projections (MAP), also referred to as iterative demeaning or the "zig-zag" algorithm (Guimaraes and Portugal 2010; Gaure 2013). Many packages employ MAP or its variants, frequently combined with accelerations (Berge 2018; Correia 2017), such as the Irons-Tuck extrapolation used by `fixest` (Irons and Tuck 1969; Bergé et al. 2026). Sections 3 and 4 introduced the fixed-effect graph and its algebra; this section turns to MAP itself and shows how that graph geometry governs its convergence rate.

MAP solves the FWL residualization problem by iterating over one fixed effect at a time. In the worker-firm-year model, MAP first subtracts worker means from the current residual, then firm means from the updated residual, then year means, and so on until convergence.

We write the FWL normal equations (see Equation 4) in block form with $D = [D_W \quad D_F \quad D_Y]$ as

$$\begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix} \begin{pmatrix} \alpha_W \\ \alpha_F \\ \alpha_Y \end{pmatrix} = \begin{pmatrix} D_{W'} W \mu \\ D_{F'} W \mu \\ D_{Y'} W \mu \end{pmatrix}. \tag{13}$$

Equivalently,

$$G_{WW}\alpha_W + C_{WF}\alpha_F + C_{WY}\alpha_Y = D_{W'} W\mu, \tag{14}$$

$$C_{(WF)'}\alpha_W + G_{FF}\alpha_F + C_{FY}\alpha_Y = D_{F'} W\mu, \tag{15}$$

$$C_{(WY)'}\alpha_W + C_{(FY)'}\alpha_F + G_{YY}\alpha_Y = D_{Y'} W\mu. \tag{16}$$

Each equation can be rearranged to express one block of effects conditional on the others. Using $C_{WF} = D_{W'} W D_F$ and $C_{WY} = D_{W'} W D_Y$ to factor $D_{W'} W$ out of the right-hand side, the first equation becomes

$$G_{WW}\alpha_W = D_{W'} W(\mu - D_F\alpha_F - D_Y\alpha_Y). \tag{17}$$

Because G_{WW} is a diagonal matrix whose entries are workers' total observation weights, solving for α_W divides each worker's weighted partial residual by its total observation weight. We apply the same rearrangement to the second equation to obtain the firm equation

$$G_{FF}\alpha_F = D_{F'}W(\mu - D_W\alpha_W - D_Y\alpha_Y), \tag{18}$$

and the year equation is analogous. Because G_{WW}, G_{FF}, and G_{YY} are all diagonal, each of the three equations is solved by computing a weighted group mean in a single pass over observations.

MAP uses this diagonal structure iteratively. Holding the other effects fixed, it updates the worker effects from the current partial residual, then repeats the same step for firms and years. Each sweep therefore cycles through the fixed-effect dimensions, subtracting the weighted group mean of the current partial residual for the factor being updated.

The cross-tabulation blocks C_{WF}, C_{WY}, and C_{FY} enter the algorithm only indirectly. For instance, the worker update is computed from the partial residual $\mu - D_F\alpha_F - D_Y\alpha_Y$, while the firm update is computed from $\mu - D_W\alpha_W - D_Y\alpha_Y$. Worker-firm, worker-year, and firm-year links are thus not solved as coupled subproblems; their effect propagates through the residual that one block update transmits to the next.

This strategy is effective when the graph is well connected. In a high-mobility worker-firm panel, many workers move across firms, so that worker and firm effects can be compared through many overlapping employment histories. A high wage at one firm can then be related to wages earned by the same workers at other firms, and a worker update rapidly alters the information available to the next firm update, and vice versa.

When mobility is sparse, sorting is strong, or one factor is nearly nested in another, MAP slows down. The information that separates worker effects from firm effects travels through movers, the edges of the worker-firm graph, and MAP picks it up only indirectly, through repeated residual updates. When those edges are few or bunched into narrow regions of the graph, each further iteration carries little fresh information. MAP still converges, but it may need many sweeps; each sweep is cheap because the diagonal block solves reduce to one-pass group means.

The same graph perspective gives a simple diagnostic for MAP difficulty in a fixed-effect structure. For any pair of fixed effects (q, r), we form the normalized cross-tabulation $H_{qr} = G_{qq}^{-\frac{1}{2}} C_{qr} G_{rr}^{-\frac{1}{2}}$ and let $\rho_{qr} = \sigma_2\left(H_{qr}\right)^2$ be the square of its largest nontrivial singular value, equivalently the largest nontrivial eigenvalue of $H_{(qr)'}H_{qr}$. The largest singular value of H_{qr} equals one on every connected component and carries no information about connectivity, so we discard it. We report the spectral gap $1 - \rho_{qr}$ in the benchmarks below. This gap measures how well connected the factor-pair graph is.[5] Gaps near zero signal sparse mobility or near-nesting, the settings in which MAP converges slowly; larger gaps indicate

[5] When the graph has more than one connected component, we compute the gap on each component and report the smallest, together with its observation share.

better connected factor-pair graphs. For the worker-firm example of Section 4, the gap is $\frac{1}{3}$.[6]

6. The Factor-Pair Schwarz Preconditioner

6.1. Preconditioners

The previous section attributed MAP's slow convergence to thin connections in the fixed-effect graph. A solver that receives this connectivity information directly should converge faster. MAP has no natural way to accept it: its update rule is fully determined by the list of absorbed factors, and the cross-tabulations enter only through the residual that one update passes to the next. Krylov solvers, by contrast, take an additional input, the preconditioner, and this input is where we place the graph structure. We therefore replace factor-by-factor demeaning via MAP with LSMR (Fong and Saunders 2011), an iterative least-squares algorithm that improves an initial guess through repeated residual corrections. The change of solver gains little by itself: a poorly conditioned Gramian G slows LSMR just as a poorly connected graph slows MAP. The core acceleration stems from building a good preconditioner, which we focus on in the remainder of this section.

How fast LSMR converges depends on the conditioning of G. When G is well conditioned, LSMR shrinks every component of the residual at a comparable rate. When G is poorly conditioned, LSMR removes some components after a few iterations, whereas components that correspond to weak links, sparse mobility, or near nesting in the fixed-effect graph shrink much more slowly; the iteration then spends most of its steps on these slow components. A preconditioner counteracts this imbalance: it changes the coordinates of the linear system so that slow and fast components decay at more similar rates, without changing the least-squares solution.

The ideal preconditioner is G^{-1}.[7] If $M^{-1} = G^{-1}$, then the preconditioned operator is

$$M^{-1}G = G^{-1}G = I. \tag{19}$$

In exact arithmetic, the Krylov iteration would then recover the solution after one correction, because the system has no slow directions left to remove. To form G^{-1} we would have to solve the fixed-effect normal equations themselves, so the identity case serves as a benchmark rather than an implementable preconditioner. A useful preconditioner must approximate enough of G^{-1} to remove the slow directions of the iteration, while its construction and repeated application must amortize over the iterations it saves.[8]

[6] In that example, $G_{WW} = \mathrm{diag}(2,2,2)$, $G_{FF} = \mathrm{diag}(3,3)$, and $C_{WF} = \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}$, so $H_{WF} = G_{WW}^{-\frac{1}{2}} C_{WF} G_{FF}^{-\frac{1}{2}} = \frac{1}{\sqrt{6}} \begin{pmatrix} 1 & 1 \\ 2 & 0 \\ 0 & 2 \end{pmatrix}$. The graph is connected, and $H_{(WF)'} H_{WF} = \frac{1}{6}\begin{pmatrix} 5 & 1 \\ 1 & 5 \end{pmatrix}$ has eigenvalues 1 and $\frac{2}{3}$. After dropping the unit eigenvalue, $\rho_{WF} = \frac{2}{3}$ and the gap is $\frac{1}{3}$.

[7] As in any model with several fixed effects, the level effects are pinned down only up to a normalization: we can add a constant to every worker effect and subtract it from every firm effect without changing the fitted values $D\alpha$, and likewise for years. The dummy-coded $G = D'WD$ and the smaller blocks inverted below are therefore not invertible until this indeterminacy is removed – one normalization for the worker-firm pair, and a second once year effects are added, as in the Section 4 example. We adopt the standard normalization within each connected set and read every inverse below as that of the resulting system. The estimated effects depend on the normalization; the residualized outcome and regressors, which are all the regression uses, do not.

[8] LSMR never forms $M^{-1}G$ explicitly, nor G itself. The iteration requires only products with D and D' and applications of M^{-1}, supplied as linear operators.

6.2. From the Block Inverse to the Diagonal Preconditioner

The block structure of G^{-1} shows which parts of the ideal inverse a feasible preconditioner should retain. For the worker-firm-year AKM model, the Gramian has the block form

$$G = \begin{pmatrix} G_{WW} & C_{WF} & C_{WY} \\ C_{(WF)'} & G_{FF} & C_{FY} \\ C_{(WY)'} & C_{(FY)'} & G_{YY} \end{pmatrix}, \tag{20}$$

with diagonal weighted-count blocks G_{WW}, G_{FF}, G_{YY} and off-diagonal cross-tabulations C_{WF}, C_{WY}, C_{FY}. Block inversion via the Schur complement shows how each off-diagonal block enters G^{-1}. For the two-factor block

$$G_2 = \begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}, \tag{21}$$

it gives the explicit formula

$$G_2^{-1} = \begin{pmatrix} G_{WW}^{-1} + G_{WW}^{-1} C_{WF} S^{-1} C_{(WF)'} G_{WW}^{-1} & -G_{WW}^{-1} C_{WF} S^{-1} \\ -S^{-1} C_{(WF)'} G_{WW}^{-1} & S^{-1} \end{pmatrix}. \tag{22}$$

$$S = G_{FF} - C_{(WF)'} G_{WW}^{-1} C_{WF}. \tag{23}$$

The formula shows that every block of G_2^{-1} depends on C_{WF} only through the Schur complement S: the cross-tabulation enters through matrix products, never as its own inverse. G_{WW} is diagonal, so G_{WW}^{-1} is a division by weighted worker counts. The Schur complement S, by contrast, is the firm-side mobility system that remains after eliminating workers. At the scale of modern worker-firm register data, solving this system is expensive: exact factorization creates many additional nonzero entries, separate connected components require separate normalizations, and weak mobility makes the remaining directions slow to resolve. S^{-1} therefore carries almost all the cost of the solve. Block inversion via Schur complements generalizes to three factors: the closed form has more terms, but every block of G^{-1} still depends jointly on the cross-tabulations C_{WF}, C_{WY}, C_{FY}.

The coarsest approximation to G^{-1} keeps only the diagonal count inverses and drops the Schur-complement corrections,

$$M_{\text{diag}}^{-1} = \operatorname{diag}(G_{WW}^{-1}, G_{FF}^{-1}, G_{YY}^{-1}). \tag{24}$$

This preconditioner is a single division by weighted level counts. It is the diagonal preconditioner used with LSMR in `FixedEffectModels.jl` (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026). Diagonal scaling encodes how many observations a level carries, but not how that level connects to the rest of the labor market. Those counts can already be useful when employment is concentrated in a few large firms, such as Novo Nordisk in Denmark or Samsung in South Korea, because the size differences alone remove an important source of scale variation. What diagonal scaling does not record is whether workers at those firms link them broadly to other firms or remain concentrated in a narrow corner of the mobility graph.

6.3. The Factor-Pair Schwarz Approximation

Additive Schwarz preconditioning adds this missing pairwise connectivity without solving the full three-factor system (Xu 1992; Toselli and Widlund 2005). It splits the fixed-effect problem into smaller overlapping pair problems. In the AKM case, one problem contains workers and firms, another contains workers and years, and a third contains firms and years. The worker-firm problem moves residual information along observed employment links; the other two pair problems do the same for worker-year and firm-year links. We then combine the three pair corrections in the full coefficient space. The preconditioner therefore gives the Krylov iteration the main pairwise channels of the Gramian, while the outer iteration handles the remaining three-way coupling.

To make the local subproblems concrete, we first consider the worker-firm pair. Its local problem is exactly the two-factor block from the Schur calculation,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}. \tag{25}$$

Figure 4 places this worker-firm pair block beside the single diagonal block solved by a factor-level MAP update.

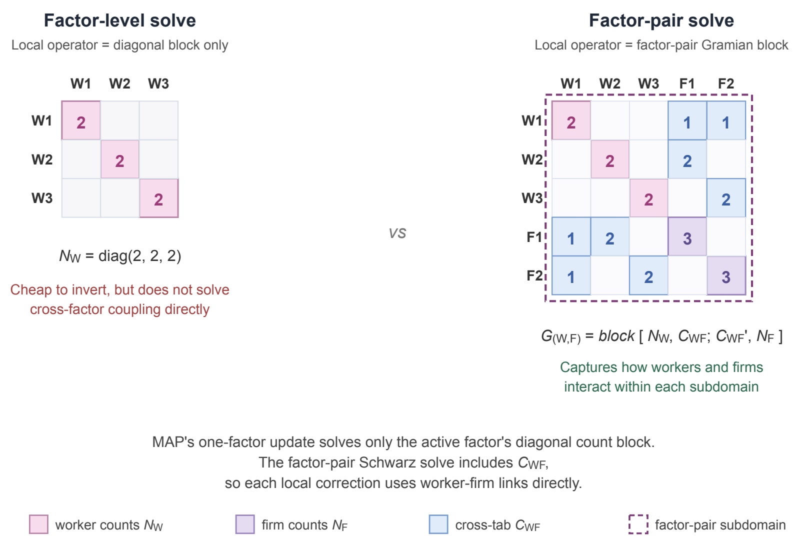

Figure 4: Local operator used by a factor-level MAP update (left) versus the factor-pair Schwarz solve (right), shown on the example worker-firm panel of Section 4. The factor-level block is the diagonal $G_{WW} = \operatorname{diag}(2, 2, 2)$. The factor-pair block adds the firm count block $G_{FF} = \operatorname{diag}(3, 3)$ and the cross-tabulation C_{WF} in its off-diagonal positions; the dashed outline marks the worker-firm subdomain on which the local Schwarz solve operates.

Its inverse carries C_{WF} through the Schur complement, so the local correction holds the worker-firm mobility geometry that diagonal scaling discards. To place this correction inside the three-factor problem, let R_{WF} select the worker and firm entries from the full coefficient vector $\alpha = [\alpha_W; \alpha_F; \alpha_Y]$; its transpose $R_{(WF)'}$ places the resulting correction back into the full vector. The diagonal matrix $\tilde{D}_{WF}$ contains the weights that split levels across the pair problems in which they appear; we call these entries partition-of-unity weights. The exact worker-firm contribution is

$$P_{WF}^{-1} = R_{(WF)'}\tilde{D}_{WF}\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}^{-1}\tilde{D}_{WF}R_{WF}. \quad (26)$$

The worker-year and firm-year contributions P_{WY}^{-1} and P_{FY}^{-1} are built the same way from G_{WW}, C_{WY}, G_{YY} and G_{FF}, C_{FY}, G_{YY}. With three factors each level appears in exactly two pair problems. Because the weights act on both sides of each pair contribution, we set them so that the squared weights on a shared level sum to one; a level in two pairs then carries $\frac{1}{\sqrt{2}}$. The exact factor-pair Schwarz preconditioner is the sum of these three contributions,

$$P^{-1} = P_{WF}^{-1} + P_{WY}^{-1} + P_{FY}^{-1}. \quad (27)$$

The three terms include the worker-firm, worker-year, and firm-year cross-tabulations separately. They do not solve the simultaneous worker-firm-year problem; that remaining coupling, together with the error introduced by splitting shared levels across pairs, is left to the outer LSMR iteration.

6.4. Approximate Pair Solves via Graph Laplacians

For P^{-1} to serve as the operator M^{-1} inside LSMR, each pair contribution must be computed without solving a large dense system. For factor pairs with few levels, we invert the pair block directly. In the example worker-firm panel of Section 4, the pair block has three worker levels and two firm levels; after the normalization of Section 6.1 removes the one free constant, the local solve is a 4×4 inversion. At the scale of modern worker-firm register data, however, a single pair can carry hundreds of thousands of levels per side, and direct inversion becomes the same kind of large linear-algebra problem the preconditioner is meant to avoid. We instead use the graph-Laplacian structure of the pair block.

For a worker-firm pair, the local Schwarz step solves the pair-Gramian system

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix} x = u, \quad (28)$$

where u is the weighted worker-firm part of the current Krylov residual. This block is not a graph Laplacian, because its off-diagonal entries C_{WF} are non-negative. A sign flip removes the obstacle. Let $T_{WF} = \operatorname{diag}(I_W, -I_F)$ flip the sign of the firm entries, so that $T_{WF}^2 = I$. Conjugation by T_{WF} gives

$$L_{WF} = T_{WF}\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}T_{WF} = \begin{pmatrix} G_{WW} & -C_{WF} \\ -C_{(WF)'} & G_{FF} \end{pmatrix}, \quad (29)$$

a weighted bipartite graph Laplacian: symmetric, with non-positive off-diagonals and zero row sums. Because $T_{WF}^2 = I$, the pair-Gramian solve follows from the Laplacian solve by the same flip on each side,

$$\begin{pmatrix} G_{WW} & C_{WF} \\ C_{(WF)'} & G_{FF} \end{pmatrix}^{-1} = T_{WF}L_{WF}^{-1}T_{WF}, \quad (30)$$

where both inverses are read as in Section 6.1: the solve fixes the free constant on each connected component by returning the zero-mean solution, and the residualized variables do not depend on this choice. A single Laplacian solve therefore yields the pair-Gramian solution: we flip the residual, apply L_{WF}^{-1}, and flip back.

For preconditioning, this local solve need not be exact. The outer LSMR iteration refines any error left by the preconditioner. We therefore approximate the Laplacian solve using sparse approximate Cholesky factorizations from the Laplacian-solver literature (Spielman and Teng 2014; Gao et al. 2025). An exact Cholesky factorization of the pair block creates fill-in: eliminating a worker inserts entries linking every pair of distinct firms that worker visited, and these entries accumulate as the elimination proceeds. In the worst case, the cost approaches the order k^3 operations and order k^2 memory of a dense factorization of a k-level system. The randomized approximate factorization avoids this growth on any graph: its cost is nearly linear in the number of observed worker-firm links, that is, linear up to logarithmic factors. After transforming this approximate Laplacian solve back to worker-firm coordinates, we write A_{WF} for the resulting approximate pair-Gramian solve.

The same construction yields A_{WY} and A_{FY} for the other two pairs. We substitute these approximate inverses into the Schwarz sum to obtain the implemented preconditioner,

$$M^{-1} = \sum_{(q,r)} R_{(qr)'} \tilde{D}_{qr} A_{qr} \tilde{D}_{qr} R_{qr}. \tag{31}$$

In the worker-firm-year case each factor receives a diagonal contribution from its two pair subdomains and each off-diagonal correction from the corresponding pair.

The approximate pair inverse introduces a second approximation, beyond the pairwise splitting. The exact P^{-1} already replaces the full three-factor inverse with pair solves; M^{-1} now applies each pair solve only approximately, through A_{qr}. Both choices shape the preconditioned search directions and nothing else: the fitted residuals are unchanged, and LSMR still solves the original fixed-effect least-squares problem to the requested tolerance.

6.5. Implementation Strategy

Figure 5 summarizes the full construction. We start with the absorbed factors and split the fixed-effect graph into overlapping factor pairs. For each pair, the local Gramian block is represented as a graph Laplacian after a sign flip; sparse approximate Cholesky solves then provide an approximate pair inverse, returned to Gramian coordinates. The pair corrections are combined with partition-of-unity weights to form the Schwarz preconditioner M^{-1}.

The outer LSMR iteration uses this preconditioner to shape its search directions (Fong and Saunders 2011; Arridge et al. 2014; Yang et al. 2024). Once constructed, the same preconditioner can be reused to residualize the outcome and every covariate, and the algorithmic details are collected in Appendix A.

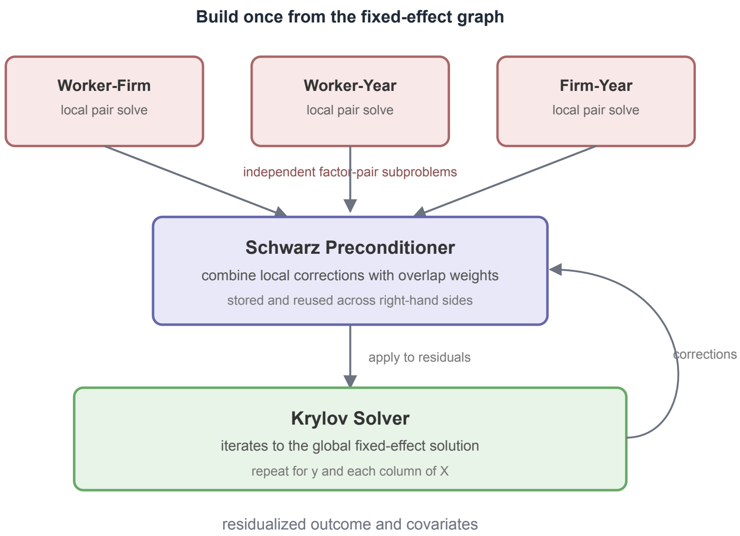

Figure 5: Summary of the factor-pair preconditioner. The fixed effects are split into overlapping factor pairs; each pair solve uses the Laplacian representation with approximate Cholesky; the resulting Schwarz preconditioner is then applied inside the outer LSMR iteration.

7. Benchmarks

Weak connectivity makes MAP's factor-by-factor updates pass information slowly through the fixed-effect graph. Factor-pair preconditioning should help most on these graphs and may add unnecessary setup cost on well-connected ones. We test this prediction on controlled synthetic designs and public benchmark datasets.

The main runtime tables use package-level regression APIs, matching the public PyFixest benchmark suite, rather than isolated demeaning kernels. Each timing covers the full regression workflow: model setup, construction of the fixed-effect representation, residualization of the outcome and covariates, and estimation of the coefficient of interest. Appendix B verifies that the preconditioned solver matches MAP to numerical tolerance and reports the memory cost of storing factor-pair information and additional solver diagnostics.

For cross-package comparisons, every backend receives the same input data but uses its own default handling of singleton or separated observations. Successful fits record the retained-observation count; failed attempts record their convergence status and error. Any shared control is stated with the relevant table. The comparisons are not constrained to a common estimation sample. The single-package mechanism experiments later in the paper do use a common prepared sample, because their purpose is to isolate the preconditioner.

The runtime tables also report the pairwise hardness statistic for the relevant factor pair and the observation share of the component attaining it. Smaller gaps indicate factor pairs on which MAP tends to converge more slowly; with three or more fixed effects, the statistic remains a pairwise diagnostic rather than a full convergence bound.

The OLS package-runtime tables compare six configurations: PyFixest MAP, PyFixest LSMR with no, diagonal, or factor-pair preconditioning, R `fixest`, and `FixedEffectModels.jl`:

Backend	Package	Algorithm
`rust-map`	PyFixest	Vanilla Rust MAP, without acceleration.
`within-off`	PyFixest	LSMR without preconditioning.
`within-diagonal`	PyFixest	LSMR with diagonal preconditioning.
`within`	PyFixest	LSMR with factor-pair Schwarz preconditioning.
`fixest`	R `fixest`	Accelerated MAP with Irons-Tuck and other optimizations (Bergé et al. 2026).
`FEM.jl`	`FixedEffectModels.jl`	Diagonally preconditioned LSMR (Fong and Saunders 2011; FixedEffectModels.jl Developers 2026).

Reported CPU times are medians over each benchmark's measured repetitions. Headline OLS uses the adaptive R1/R2/R3 rule; experiments with a fixed count use three repetitions. All runs use an Apple M4 Mac mini with 10 CPU cores and 16 GB of memory running macOS 15.3.1.

We do not include `reghdfe` (Correia 2014; 2017) directly in the benchmark tables because Stata is not open source and we lack a license. `reghdfe` is a mature accelerated-MAP implementation; algorithmically, it belongs to the same family as `fixest`'s accelerated MAP.

7.1. Runtime Benchmarks

The implementations use different convergence criteria, so their nominal tolerances are not directly comparable. The first comparisons report package defaults. The achieved-precision experiment later in this section evaluates every method against the same external error measures.

The main OLS benchmarks use synthetic AKM-style panels with one million observations, one covariate, ten periods, and worker, firm, and year fixed effects. They vary the worker-firm graph in two ways while holding the rest of the data-generating process fixed. The mobility designs progressively reduce the number of workers who change firms. The sorting designs instead concentrate moves within groups of firms, creating weakly connected blocks even though workers continue to move. Appendix B reports further synthetic designs and the empirical HDFE benchmarks assembled by Sergio Correia.

7.1.1. Worker Mobility

Lower worker mobility leaves fewer workers connecting multiple firms. MAP still updates one fixed-effect dimension at a time, so information about firm effects moves more slowly through the iteration. The factor-pair preconditioner uses the worker-firm links directly and should become relatively more competitive as mobility falls.

Mobility benchmark ($n = 1$M).

Scenario	Gap (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
`akm_mobility_1`	0.41 (1.00)	0.388s	2.45s	0.350s	0.543s	0.483s	0.330s
`akm_mobility_2`	0.0769 (1.00)	1.13s	3.60s	0.453s	0.551s	0.894s	0.585s
`akm_mobility_3`	3.67×10^{-3} (0.87)	18.0s	capped (0/3)	1.02s	0.444s	2.58s	2.03s
`akm_mobility_4`	1.07×10^{-3} (0.61)	54.5s	capped (0/3)	1.37s	0.404s	5.83s	2.93s
`akm_mobility_5`	1.10×10^{-4} (0.52)	capped (0/3)	capped (0/3)	2.10s	0.382s	6.53s	3.37s
`akm_mobility_6`	4.82×10^{-5} (0.29)	capped (0/3)	capped (0/3)	3.10s	0.365s	7.25s	4.29s

Note: Entries are median full-regression wall-clock times in seconds at package-default settings. A value $t\left(\frac{k}{n}\right)$ is the median among k successful calls when fewer than n planned calls converge; a dash means that no complete result is recorded. `capped (0/n)` and `failed (0/n)` distinguish iteration limits from other errors. The AKM-style panels have 1M observations, one covariate, and worker, firm, and year fixed effects. Lower rows reduce worker mobility within a 10-period panel. Gap is $1 - \rho_{WF}$ for the worker-firm pair; parentheses give the observation share of the component attaining the gap.

The worker-firm gap falls from 0.41 to 4.82×10^{-5} as mobility declines, although different connected components attain the reported gap. In the first two designs, all six configurations finish in at most 3.60 seconds. From the third design onward, PyFixest MAP slows sharply or reaches its cap, and LSMR without preconditioning reaches its cap. Factor-pair LSMR falls from 0.543 seconds in the first design to 0.365 seconds in the last and is the fastest PyFixest LSMR configuration in the four lowest-mobility designs.

7.1.2. Sorting Among Movers

Stronger worker-firm sorting produces weakly connected blocks: movers increasingly connect firms within the same group rather than linking different groups. MAP should therefore slow down again, because information about firm effects crosses groups only through a small number of movers. The preconditioned method should be less sensitive, since its factor-pair solves use the worker-firm graph directly.

Sorting benchmark ($n = 1$M).

Scenario	Gap (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
`akm_sorting_1`	0.0129 (0.97)	5.63s	capped (0/3)	0.704s	0.441s	1.29s	1.32s
`akm_sorting_2`	5.76×10^{-3} (0.96)	10.2s	capped (0/3)	0.905s	0.485s	1.77s	1.68s
`akm_sorting_3`	6.55×10^{-3} (0.96)	10.00s	capped (0/3)	1.15s	0.488s	1.73s	1.62s
`akm_sorting_4`	4.21×10^{-3} (0.96)	16.1s	capped (0/3)	0.988s	0.501s	2.31s	1.75s
`akm_sorting_5`	2.19×10^{-3} (0.96)	28.6s	capped (0/3)	0.997s	0.487s	2.59s	1.73s

Note: Entries are median full-regression wall-clock times in seconds at package-default settings. A value $t\left(\frac{k}{n}\right)$ is the median among k successful calls when fewer than n planned calls converge; a dash means that no complete result is recorded. `capped (0/n)` and `failed (0/n)` distinguish iteration limits from other errors. The AKM-style panels have 1M observations, one covariate, and worker, firm, and year fixed effects. Lower rows raise sorting among movers. Gap is $1 - \rho_{WF}$ for the worker-firm pair; parentheses give the observation share of the component attaining the gap.

The gap ranges from 0.0129 to 0.00219 across these designs. Different components attain the reported gap in different rows, so its ordering is not monotone. MAP slows from 5.63 to 28.6 seconds as sorting increases. LSMR without preconditioning reaches its default cap in every row. Diagonal LSMR takes 0.704 to 1.15 seconds, whereas factor-pair LSMR takes 0.441 to 0.501 seconds and is the fastest method in every sorting design.

7.1.3. When Preconditioner Setup Dominates

The AKM designs vary connectivity gradually. The public `fixest` benchmark suite gives a sharper comparison between a dense, well-connected design and a sparse, nearly nested one (Bergé et al. 2026). Both contain 10 million observations, one covariate, and worker, firm, and year fixed effects.

Simple and difficult `fixest` designs (10M observations, 3 FE).

Design	Gap (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
simple (well-connected)	0.857 (1.00)	2.58s	2.69s	2.64s	11.5s	2.64s	2.16s
difficult (near-nested)	1.67×10^{-7} (1.00)	337.2s	11.0s	capped (0/3)	4.45s	63.5s	27.8s

Note: Entries are median full OLS regression wall-clock times in seconds at package-default settings. An unreported initial timing selects 20 measured calls below 1 second, 7 from 1 to 10 seconds, and 3 otherwise. A value $t\left(\frac{k}{n}\right)$ is the median among k successful calls when fewer than n planned calls converge; a dash means that no complete result is recorded. `capped (0/n)` and `failed (0/n)` identify unsuccessful cells. Both designs have 10M observations, one covariate, and worker, firm, and year fixed effects. Gap is $1 - \rho_{WF}$ for the worker-firm pair, measured on the 1M-observation version of the same DGP family; the 10M runs use the same simple/difficult graph construction.

On the dense design, all methods except factor-pair LSMR finish in 2.16 to 2.69 seconds; factor-pair LSMR takes 11.5 seconds because its setup cost is not recovered in one fit. On the near-nested design, factor-pair

LSMR takes 4.45 seconds, compared with 11.0 seconds without preconditioning, 27.8 seconds for FEM.jl, 63.5 seconds for `fixest`, and 337.2 seconds for PyFixest MAP. PyFixest diagonal LSMR reaches its default cap on that design. The standalone setup diagnostic separates construction from the subsequent solve.

7.1.4. Iterations After Setup

The iteration counts separate construction from the subsequent solves. The table uses the 100,000-observation versions of the same simple and difficult designs.

Iterations on the simple and difficult designs (100K observations).

Design	`map` (sweeps)	`off`	`diagonal`	`additive`
simple	14	37	16	14
difficult	8159	249	180	22

Note: MAP is reported in full sweeps over the fixed-effect dimensions. The LSMR columns report Krylov iterations. A MAP sweep and an LSMR iteration are different units and should not be compared across columns. Each LSMR count is the median of three solves with two right-hand sides.

Among the LSMR configurations, the additive preconditioner needs 14 iterations on the simple design and 22 on the difficult one. The diagonal count rises from 16 to 180, while unpreconditioned LSMR rises from 37 to 249. Once the additive preconditioner has been built, the difficult design requires only eight more Krylov iterations than the simple design. The preconditioned solve is short in both cases; setup makes `within` unattractive on the simple one-shot regression.

7.2. Runtime and Achieved Precision

The stopping rules attach tolerance to different quantities. PyFixest MAP defaults to 10^{-6} and stops when every observation-level residual changes by less than that amount after a full pass over the fixed-effect dimensions. R's `fixest` also defaults to 10^{-6}, but applies the threshold to successive fixed-effect coefficient iterates. It accepts either a small absolute change or a small scaled relative change, and periodically makes the same comparison for the residual sum of squares. `FixedEffectModels.jl` passes its default tolerance of 10^{-6} as both LSMR stopping tolerances. `within` defaults to 10^{-8} and stops when either the estimated least-squares residual is at most 10^{-8} times the norm of the right-hand side, or the scaled normal-equation residual falls below 10^{-8}. The latter is $\|A^T r \frac{\|_2}{\|A\|_F \ \|r\|_2}$, using LSMR's running norm estimates (Fong and Saunders 2011).

Figure 6 compares the methods on a common accuracy scale. It uses mobility designs 1, 3, and 5. Before timing, we recursively remove singletons once from each one-million-observation design and pass the resulting rows to every method. We then vary each package's own tolerance, use a common 10,000-iteration cap, and time three fits at every setting. The tight reference uses factor-pair LSMR at tolerance 10^{-14}. The package defaults for the iteration cap are 10,000 for PyFixest MAP, `fixest`, and `FixedEffectModels.jl`, and 1,000 for `within`.

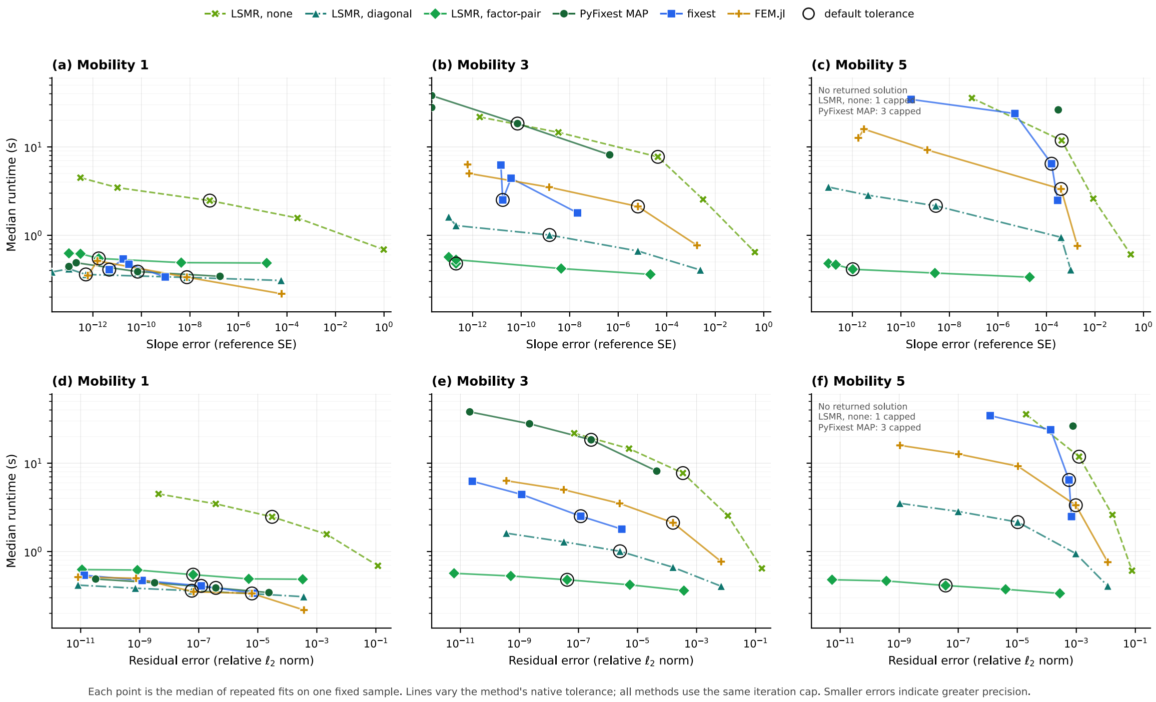

Figure 6: Runtime against achieved precision on three AKM mobility designs. Each point is the median of three fits on the same recursively singleton-pruned sample for that design; pruning is completed before the timed fits. Lines vary a package's requested tolerance, while the horizontal axis reports achieved error. The top row measures coefficient error as $\frac{|\hat{\beta}-\hat{\beta}^\star|}{\mathrm{SE}(\hat{\beta}^\star)}$. The bottom row measures final residual error as $\frac{\|r-r^\star\|_2}{\|r^\star\|_2}$. Both use the tight factor-pair LSMR reference. Error decreases from left to right. Circled points use each package's default tolerance. All runs have a 10,000-iteration cap. Settings that return no solution have no horizontal coordinate and are omitted; the annotations report these settings.

7.3. Amortizing the Preconditioner

The additive method first constructs the factor-pair blocks and their approximate factorizations, then applies them during the LSMR solve. Construction is paid once for a fixed set of observations, weights, and fixed-effect identifiers.

7.3.1. Setup Cost Across Connectivity

We time these two stages separately on the AKM mobility designs. The two-factor specification absorbs worker and firm effects; the three-factor specification also absorbs year effects. Each cell is the median of five standalone solves at one million observations, with the same outcome and covariate used in both specifications.

Additive setup and solve time across AKM connectivity.

Scenario	Gap (share)	2 FE setup	2 FE solve	3 FE setup	3 FE solve
`akm_mobility_1`	0.41 (1.00)	0.119s	0.323s	0.123s	0.239s
`akm_mobility_2`	0.0769 (1.00)	0.125s	0.285s	0.129s	0.264s
`akm_mobility_3`	3.67×10^{-3} (0.87)	0.039s	0.150s	0.047s	0.254s
`akm_mobility_4`	1.07×10^{-3} (0.61)	0.053s	0.131s	0.057s	0.250s
`akm_mobility_5`	1.10×10^{-4} (0.52)	0.057s	0.123s	0.067s	0.231s
`akm_mobility_6`	4.82×10^{-5} (0.29)	0.067s	0.057s	0.073s	0.176s

Note: Times are seconds. Each cell is the median of five runs at tolerance 10^{-12}. The two-factor specification absorbs worker and firm effects; the three-factor specification adds year effects. The worker-firm gap and component share are the same diagnostics used in the mobility benchmark.

Setup is cheaper in the low-mobility designs. With two fixed effects, median setup declines from 0.119 seconds in `akm_mobility_1` to 0.067 seconds in `akm_mobility_6`; with three fixed effects, it falls from 0.123 to 0.073 seconds. Solve time falls as well, from 0.323 to 0.057 seconds with two effects and from 0.239 to 0.176 seconds with three. The low-mobility worker-firm graph has fewer cross-firm links to store and factorize. This is why the additive runtime in Figure 1 can decline while MAP and diagonal LSMR slow down.

7.3.2. Ten Regressions on the Same Fixed Effects

Many empirical projects estimate several specifications on the same sample and fixed effects. The factor-pair objects can then be constructed once and reused. We measure ten sequential regressions on both one-million-observation `fixest` designs, using the same outcome and ten covariates for all three policies. One additive policy constructs a new preconditioner for every regression; the other keeps one preconditioner for all ten.

Ten regressions with rebuilt and cached preconditioners.

Design	Policy	Setup	Solve	Total	Speedup
simple	Diagonal	0.052s	0.640s	0.692s	1.0x
	Additive, rebuilt	2.89s	1.56s	4.46s	0.2x
	Additive, cached	0.283s	1.55s	1.83s	0.4x
difficult	Diagonal	0.050s	50.3s	50.3s	1.0x
	Additive, rebuilt	0.434s	1.34s	1.77s	28.4x
	Additive, cached	0.040s	1.19s	1.23s	41.1x

Note: Medians over three repetitions for each design at 1M observations, with worker, firm, and year fixed effects. Each regression residualizes the common outcome and one covariate at tolerance 10^{-12}. Setup and solve columns sum time across all ten calls. Speedup is relative to diagonal preconditioning. Diagonal and rebuilt additive construct a solver for every call; cached additive constructs one solver.

Even across ten regressions, the additive preconditioner is slower on the simple design. Diagonal preconditioning takes 0.692 seconds. Rebuilding the additive preconditioner takes 4.46 seconds, while caching lowers this to 1.83 seconds. Avoiding nine constructions is not enough: the cached additive path is still more than twice as slow as diagonal preconditioning on this well-connected graph.

On the difficult design, diagonal preconditioning takes 50.3 seconds for the ten regressions. Rebuilding the additive preconditioner lowers this to 1.77 seconds, and caching lowers it further to 1.23 seconds. Caching reduces additive setup time from 0.434 to 0.040 seconds; the repeated solves take 1.34 seconds with rebuilding and 1.19 seconds with caching.

7.4. Poisson and Other GLMs

Faster fixed-effect solves matter especially when an estimator requires several of them, as in generalized linear models fit by iteratively reweighted least squares (IRLS). Each IRLS step fits a weighted least squares problem in which the response and covariates are demeaned against the fixed effects (Correia et al. 2020; Stammann 2018). A single GLM fit therefore calls the demeaning routine repeatedly. The fixed-effect incidence graph stays the same, but the weights change between calls, including in `ppmlhdfe`, Poisson fixed-effect estimators, and other IRLS-based GLM implementations.

PyFixest currently reuses the preconditioner built during the first weighted demeaning call. This avoids a new factorization at every IRLS step, but later problems use a preconditioner based on earlier weights.

Reuse saves construction time and may require more LSMR iterations. Rebuilding may reduce the iteration count but incurs construction cost at every step. The table reports the current PyFixest reuse behavior.

The PPML comparison uses the simple-versus-difficult DGPs from the `fixest` benchmark suite (Bergé et al. 2026) at $n = 1\text{M}$ observations, with one covariate and worker, firm, and year fixed effects. The compared backends are R `fixest`'s `fepois`, `GLFixedEffectModels.jl`, and two PyFixest `fepois` paths: the default unpreconditioned `rust-map` and `within` with PyFixest's current reuse policy. Packages apply their default rules for separated observations, and each backend receives the same 100-iteration outer IRLS limit.

Poisson benchmarks (1M observations, one covariate).

Design	FE	PyFixest MAP	fixest	GLFEM.jl	PyFixest LSMR factor-pair
simple (well-connected)	3	7.86s	4.72s	5.76s	9.25s
difficult (near-nested)	3	capped (0/3)	439.4s	129.8s	5.43s

Note: Entries are median full `fepois` wall-clock times in seconds over three calls at $n = 1\text{M}$ with one covariate and three fixed effects. `fixest` is R `fixest::fepois`; `rust-map` and `within` use PyFixest `fepois`; and `GLFEM.jl` is `GLFixedEffectModels.jl`. The `within` column uses PyFixest's current policy of reusing the first factor-pair preconditioner as the IRLS weights change. Each package applies its default separation handling. A value $t\left(\frac{k}{3}\right)$ is the median among k successful calls; `capped (0/3)` identifies an iteration limit and `failed (0/3)` another error.

On the simple design, all four paths finish in under ten seconds. `fixest` takes 4.72 seconds, `GLFEM.jl` 5.76, `rust-map` 7.86, and `within` 9.25. Runtime differences are much larger on the difficult design. `rust-map` does not converge within the iteration cap, `GLFEM.jl` takes 129.8 seconds, and `fixest` takes 439.4 seconds. `within` finishes in 5.43 seconds. Sparse worker-firm coupling slows MAP, while the factor-pair blocks encode those links directly.

8. Software

The solver studied in this paper is available as open-source software through the `within` project (within Developers 2026). The computational core is implemented in Rust and exposed through Rust, Python, and R interfaces; each language binding invokes the same underlying solver. All three interfaces call the same Rust implementation.

Interface	Package	Registry	Install
Rust	`within`	crates.io	`cargo add within`
Python	`within-py`	PyPI	`pip install within-py`
R	`withinr`	py-econometrics R-universe	`install.packages("withinr", repos = "https://py-econometrics.r-universe.dev")`

The Rust crate exposes the lower-level solver together with its configuration types. The Python and R packages provide solver-level `solve` and `solve_batch` APIs for residualizing one or several right-hand sides. The algorithm is also available as a demeaning backend for PyFixest (PyFixest Developers 2026).

The following Python example demeans an outcome and two covariates against worker and firm fixed effects, then runs the FWL regression on the demeaned variables:

```python
import numpy as np
from within import solve_batch
# Worker and firm identifiers as a column-major uint32 array
n = 100_000
categories = np.asfortranarray(np.column_stack([
    np.random.randint(0, 5_000, n).astype(np.uint32),
    np.random.randint(0,   500, n).astype(np.uint32),
]))
# Outcome and covariates
beta = np.array([1.0, -2.0])
X = np.random.randn(n, 2)
y = X @ beta + np.random.randn(n)
# Residualize y and X jointly; the preconditioner is reused across columns
res = solve_batch(categories, np.column_stack([y, X]))
y_tilde, X_tilde = res.demeaned[:, 0], res.demeaned[:, 1:]
# FWL on the demeaned variables
beta_hat = np.linalg.lstsq(X_tilde, y_tilde, rcond=None)[0]
```

9. Conclusion

The benchmarks show that solver rankings depend on graph connectivity. MAP is difficult to outperform on dense, well-connected graphs because its sweeps are cheap and factor-pair construction adds overhead. With low mobility, strong sorting, or near-nested effects, MAP passes information slowly and the factor-pair preconditioner is often much faster.

Factor-pair construction becomes cheaper in the low-mobility AKM designs and can be paid once across regressions that share the same sample, fixed effects, and weights. Caching lowers additive runtime on both simple and difficult designs, but only the difficult design is faster than diagonal preconditioning. There, caching reduces additive setup from 0.434 to 0.040 seconds and total time from 1.77 to 1.23 seconds. PPML calls the demeaning routine repeatedly because every IRLS step solves a new weighted problem. In our benchmark, PyFixest reuses the first preconditioner as the weights change. The results support factor-pair preconditioning when the gap diagnostic is small, when a fit is unexpectedly slow, or when an estimator requires several fixed-effect solves.

Appendix A: Factor-Pair Schwarz Algorithm

Algorithm 1 gives the implementation corresponding to the construction summarized in Figure 5.

Algorithm 1. Factor-Pair Schwarz Preconditioner

Inputs

- Observation-level factor codes for Q absorbed dimensions.
- Diagonal weights W and a local solver configuration.
- Krylov residual r in coefficient space.

Preconditioner setup

- Enumerate all unordered factor pairs (q, r) with $q < r$.
- For each pair, build weighted count blocks G_{qq}, G_{rr} and the weighted cross-tabulation C_{qr}.
- Split the induced bipartite graph into connected components and create one Schwarz subdomain s per component.
- If fixed-effect level j appears in c_j subdomains, store the partition weight $\omega_j = \frac{1}{\sqrt{c_j}}$.
- For each subdomain, sign-flip one side to obtain a local Laplacian, project the local right-hand side off the component constant, and build a zero-mean local solve.
- Use a Schur-complement local solver: small reduced systems are solved directly, while larger reduced symmetric diagonally dominant (SDD) or Laplacian systems are solved with randomized approximate Cholesky.

Krylov application

- Initialize $z = 0$.
- For each subdomain s, form $h_s = \tilde{D}_s R_s r$.
- Compute the approximate local correction $u_s \approx A_s h_s$ on the normalized subspace.
- Accumulate $z \leftarrow z + R_{s'} \tilde{D}_s u_s$.
- Return $z = M^{-1} r$.

Appendix B: Benchmarks and Diagnostics

This appendix reports additional synthetic and real-data benchmarks, memory use, and numerical equivalence. Each table is generated from the recorded benchmark output by `scripts/paper_results.py`; none of the numbers are entered by hand.

More Benchmarks

Standard Synthetic Benchmarks: Correia Collection

The controlled AKM benchmarks in Section 7 vary mobility and sorting directly. The Correia synthetic cases are public and reproducible.

The Correia HDFE benchmark collection spans a broader set of graph shapes: complete bipartite matching, uniform random matching with varying degrees of connectivity, assortative matching, and a small path-like design. They provide an independent public reference set for comparing the same software backends outside the AKM generator.

Correia synthetic benchmarks.

Dataset	Gap (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
`synthetic-complete`	1.00 (1.00)	0.111s	–	–	0.227s	0.088s	0.047s
`synthetic-uniform-easy`	0.651 (1.00)	0.170s	–	–	0.178s	0.165s	0.068s
`synthetic-uniform-hard`	0.184 (1.00)	0.362s	–	–	0.951s	0.359s	0.233s
`synthetic-uniform-harder`	0.0249 (1.00)	0.748s	–	–	0.560s	1.18s	0.288s
`synthetic-assortative`	1.33×10^{-3} (0.70)	13.0s	–	–	0.379s	1.37s	1.19s

Note: Entries are median full OLS regression wall-clock times in seconds over three calls at package-default settings. Each package applies its default singleton treatment; the gap is $1 - \rho$ for the `id1-id2` pair after the singleton pruning used by the hardness diagnostic, and parentheses give the observation share of the component attaining the gap. A value $t\left(\frac{k}{3}\right)$ is the median among k successful calls; a dash means that no complete result is recorded. `capped (0/3)` and `failed (0/3)` distinguish iteration limits from other errors. `synthetic-zigzag` is omitted because the default MAP backends reach their 10,000-iteration demeaning caps.

The synthetic collection covers complete, uniform, assortative, and path-like matching patterns. The no-preconditioner and diagonal LSMR results will be added once the package-default Correia run is complete; the unfilled cells are not used for rankings.

Standard Real-Data Benchmarks: Correia Collection

The Correia collection also includes real benchmark data. Synthetic data sets match the overall shape of empirical co-occurrence graphs but smooth away irregularities that often drive runtime: a few units that appear far more often than the rest, thin connections between otherwise dense groups, many small disconnected pieces, and interactions between identifiers that go beyond a single two-way pair. The empirical datasets contain these features and therefore test the solvers in conditions that controlled DGPs only approximate.

Correia real-data benchmarks.

Dataset	Gap (share)	PyFixest MAP	PyFixest LSMR none	PyFixest LSMR diagonal	PyFixest LSMR factor-pair	fixest	FEM.jl
`credit`	0.402 (1.00)	0.198s	–	–	0.213s	0.155s	0.094s
`soccer`	0.889 (1.00)	0.034s	–	–	0.056s	0.014s	0.010s
`enron`	7.04×10^{-3} (0.99)	2.95s	–	–	0.398s	0.537s	0.316s
`github`	6.30×10^{-4} (0.32)	capped (0/3)	–	–	0.312s	1.43s	1.44s
`patents`	5.18×10^{-4} (0.95)	capped (0/3)	–	–	0.384s	1.05s	0.582s
`workers`	2.74×10^{-4} (0.63)	capped (0/3)	–	–	0.284s	1.98s	1.45s
`schools`	2.21×10^{-3} (1.00)	5.79s	–	–	0.238s	0.825s	0.495s
`directors`	5.12×10^{-4} (0.30)	capped (0/3)	–	–	0.339s	0.280s	0.459s

Note: Entries are median full OLS regression wall-clock times in seconds over three calls at package-default settings. Each package applies its default singleton treatment; the gap is $1 - \rho$ for the `id1`-`id2` pair after the singleton pruning used by the hardness diagnostic, and parentheses give the observation share of the component attaining the gap. A value $t\left(\frac{k}{3}\right)$ is the median among k successful calls; a dash means that no complete result is recorded. `capped (0/3)` and `failed (0/3)` distinguish iteration limits from other errors. A small gap in a small component, as for `directors`, need not make the full sample difficult for MAP.

The real-data collection includes irregular components that are not captured by one pairwise statistic. The `directors` component attaining the smallest reported gap covers 30 percent of the observations. The no-preconditioner and diagonal LSMR columns await the package-default Correia run; the gap remains a diagnostic rather than a selection rule.

Memory Use

MAP uses little memory: a sweep needs only the current residuals and per-level group sums. A factor-pair preconditioner, by contrast, must retain the pair structure between iterations. We therefore measure how much additional memory the preconditioned Krylov solver requires relative to MAP.

We record peak resident set size (peak RSS), the largest amount of physical memory used by the process during a run, on the simple and difficult fixed-effect DGPs. The main storage terms are the data matrix, the fixed-effect encodings, and the reusable factor-pair objects. We do not compare `fixest` and `FixedEffectModels.jl` here, because peak RSS across languages also reflects R and Julia runtime overhead, data-loading choices, garbage collection, and package internals. We compare the preconditioned Rust backend with Rust MAP inside the same Python package. The surrounding regression code is shared, leaving the demeaning strategy as the relevant difference. Both backends run in isolated processes and report peak RSS through `ru_maxrss`.

Memory footprint (3 FE, one covariate).

Design	Gap (share)	PyFixest MAP	PyFixest LSMR factor-pair
100K observations			
simple (well-connected)	0.857 (1.00)	354 MiB	409 MiB
difficult (near-nested)	1.30×10^{-3} (1.00)	359 MiB	408 MiB
1M observations			
simple (well-connected)	0.857 (1.00)	833 MiB	1,101 MiB
difficult (near-nested)	1.67×10^{-5} (1.00)	821 MiB	949 MiB

Note: Entries are peak resident-set sizes (MiB), measured from isolated Python processes. The table compares PyFixest MAP with PyFixest factor-pair LSMR in full OLS calls with one covariate and worker, firm, and year fixed effects. The two DGPs are representative probes rather than a test of design-specific memory behavior. Gap is $1 - \rho_{WF}$ for the worker-firm pair in the corresponding generated design; parentheses give the observation share of the component attaining the gap.

At 100K observations, the preconditioner adds 49–55 MiB. At 1M observations the overhead is 128–268 MiB, but it remains modest relative to the full panel data footprint. The additional storage holds factor-pair co-occurrences, partition weights, and local approximate Cholesky factors.

Numerical Equivalence

Numerical agreement is a separate diagnostic because MAP and LSMR-style routines use different convergence checks, residual norms, stopping thresholds, and iteration caps. Two correct implementations can therefore return coefficients that agree only up to their tolerances.

We use the 100K-observation simple and difficult data generating processes from the `fixest` benchmarks as diagnostic cases. All methods should agree closely on the simple design. The difficult design is poorly conditioned, so small differences in stopping rules are more likely to appear. The worker-firm gap is approximately 0.857 in the simple design and 0.00130 in the difficult design. The coefficient-agreement table compares one regression coefficient across the four backends.

Coefficient agreement (100K observations, 3 FE, one covariate).

Design	Configuration	$\hat{\beta}_1$	Absolute difference
simple	PyFixest MAP	1.00463872	–
	PyFixest LSMR factor-pair	1.00463872	0
	fixest	1.00463872	1.1×10^{-15}
	FEM.jl	1.00463872	2.8×10^{-14}
difficult	PyFixest MAP	0.99953355	–
	PyFixest LSMR factor-pair	0.99953387	3.2×10^{-7}
	fixest	0.99953387	3.2×10^{-7}
	FEM.jl	0.99953385	3.0×10^{-7}

Note: Each entry is one full OLS regression call at package-default settings. $\hat{\beta}_1$ is the slope coefficient on `x1`; the final column is $\left|\hat{\beta}_1 - \hat{\beta}_{1,\text{ rust-map}}\right|$. The dash in the `rust-map` row marks the reference estimate. `within` is PyFixest LSMR with factor-pair preconditioning.

On the simple design, the largest coefficient difference is 2.8×10^{-14}. On the difficult design it is 3.2×10^{-7}. The methods use different stopping checks, so exact agreement is not expected on the poorly conditioned design.

References

Abowd, John M., Francis Kramarz, and David N. Margolis. 1999. "High Wage Workers and High Wage Firms." *Econometrica* 67 (2): 251–333. https://doi.org/10.1111/1468-0262.00020.

Arridge, Simon R., Marta M. Betcke, and Lauri Harhanen. 2014. "Iterated Preconditioned LSQR Method for Inverse Problems on Unstructured Grids." *Inverse Problems* 30 (7): 75009. https://doi.org/10.1088/0266-5611/30/7/075009.

Berge, Laurent. 2018. *Efficient Estimation of Maximum Likelihood Models with Multiple Fixed-Effects: The R Package Fenmlm.* Technical Report Nos. 2018–13.

Bergé, Laurent R., Kyle Butts, and Grant McDermott. 2026. "Fixest: A Fast and Feature-Rich Framework for Econometric Estimations in R.". https://doi.org/10.48550/arXiv.2601.21749.

Correia, Sergio. 2014. "Reghdfe: Stata Module to Perform Linear or Instrumental-Variable Regression Absorbing Any Number of High-Dimensional Fixed Effects.". https://github.com/sergiocorreia/reghdfe.

Correia, Sergio. 2017. *Linear Models with High-Dimensional Fixed Effects: An Efficient and Feasible Estimator.* Technical report. https://hdl.handle.net/10.2139/ssrn.3129010.

Correia, Sergio, Paulo Guimarães, and Tom Zylkin. 2020. "Fast Poisson Estimation with High-Dimensional Fixed Effects." *The Stata Journal* 20 (1): 95–115. https://doi.org/10.1177/1536867X20909691.

FixedEffectModels.jl Developers. 2026. "Fixedeffectmodels.jl: Fast Estimation of Linear Models with High Dimensional Categorical Variables.". https://github.com/FixedEffects/FixedEffectModels.jl.

Fong, David Chin-Lung, and Michael Saunders. 2011. "LSMR: An Iterative Algorithm for Sparse Least-Squares Problems." *SIAM Journal on Scientific Computing* 33 (5): 2950–71. https://doi.org/10.1137/10079687X.

Frisch, Ragnar, and Frederick V. Waugh. 1933. "Partial Time Regressions as Compared with Individual Trends." *Econometrica* 1 (4): 387–401. https://doi.org/10.2307/1907330.

Gao, Yuan, Rasmus Kyng, and Daniel A. Spielman. 2025. "AC(k): Robust Solution of Laplacian Equations by Randomized Approximate Cholesky Factorization." *SIAM Journal on Scientific Computing*,. https://rasmuskyng.com/papers/GKS25.pdf.

Gaure, Simen. 2013. "OLS with Multiple High Dimensional Category Variables." *Computational Statistics & Data Analysis* 66 : 8–18. https://doi.org/10.1016/j.csda.2013.03.024.

Goldsmith-Pinkham, Paul. 2026. *Tracking the Credibility Revolution across Fields.* Technical report.

Guimaraes, Paulo, and Pedro Portugal. 2010. "A Simple Feasible Procedure to Fit Models with High-Dimensional Fixed Effects." *The Stata Journal* 10 (4): 628–49. https://doi.org/10.1177/1536867X1101000406.

Irons, Bruce M., and Richard C. Tuck. 1969. "A Version of the Aitken Accelerator for Computer Iteration." *International Journal for Numerical Methods in Engineering* 1 (3): 275–77. https://doi.org/10.1002/nme.1620010306.

Lovell, Michael C. 1963. "Seasonal Adjustment of Economic Time Series and Multiple Regression Analysis." *Journal of the American Statistical Association* 58 (304): 993–1010. https://doi.org/10.1080/01621459.1963.10480682.

PyFixest Developers. 2026. "Pyfixest: Fast High-Dimensional Fixed Effects Regression in Python.". https://github.com/py-econometrics/pyfixest.

Spielman, Daniel A., and Shang-Hua Teng. 2014. "Nearly Linear Time Algorithms for Preconditioning and Solving Symmetric, Diagonally Dominant Linear Systems." *SIAM Journal on Matrix Analysis and Applications* 35 (3): 835–85. https://doi.org/10.1137/090771430.

Stammann, Amrei. 2018. "Fast and Feasible Estimation of Generalized Linear Models with High-Dimensional K-Way Fixed Effects.". https://doi.org/10.48550/arXiv.1707.01815.

Toselli, Andrea, and Olof Widlund. 2005. *Domain Decomposition Methods: Algorithms and Theory*. Springer. https://doi.org/10.1007/b137868.

within Developers. 2026. "Within: High-Performance Fixed-Effects Solver for Econometric Panel Data.". https://github.com/py-econometrics/within.

Xu, Jinchao. 1992. "Iterative Methods by Space Decomposition and Subspace Correction." *SIAM Review* 34 (4): 581–613. https://doi.org/10.1137/1034116.

Yang, Mei, Gul Karaduman, and Ren-Cang Li. 2024. "Flexible Modified LSMR for Least Squares Problems.".